Network Packet Sniffer

A high-performance raw-socket analyzer with a live dashboard

Project Documentation

Architecture, protocols, layers, UI walkthrough, build & run

Generated 2026-05-26

Repository: yedidiaSch/raw_socket_sniffer - Branch: fix/runtime-stability

1. Introduction

1.1 What this project is

This project is a network packet sniffer for Linux written in C, paired with a Python text-mode dashboard that visualizes the captured traffic in real time. The C side does the heavy lifting: it opens a raw socket bound directly to a network interface, asks the kernel for a shared ring buffer (zero-copy capture), decodes each frame layer-by-layer, and streams structured metadata over UDP to the Python frontend. The Python side listens, aggregates statistics, and renders a colorful live table using the rich library.

1.2 Why a packet sniffer?

Packet sniffers (or 'network analyzers') reveal what really crosses the wire. They are essential for:

- Network troubleshooting: 'why can't this app reach that server?'
- Security auditing: detecting unencrypted credentials, rogue devices, suspicious flows.
- Wireless research: capturing 802.11 management frames, WPA handshakes (with permission), measuring signal strength.
- Learning: seeing TCP three-way handshake, DNS queries, ARP traffic with your own eyes makes networking concrete.

1.3 Who this document is for

It assumes you can read C and a little Python and have used a terminal, but it does NOT assume deep networking knowledge. Every protocol parsed by this tool is explained from first principles before showing the code that handles it.

> *NOTE: Capturing packets you do not own may be illegal. Use this tool on networks you control or have explicit permission to analyze.*

2. Networking Stack Primer

2.1 Layers and encapsulation

Networks are built in layers. Each layer adds a header (and sometimes a footer) wrapping the payload from the layer above. A web request travels down the stack on the sender, across the wire, and back up the stack on the receiver.

Encapsulation of a typical web request

```
+-----------------------------------------------------------+
| Ethernet header | IP header | TCP header | HTTP payload    |
+-----------------------------------------------------------+
 14 bytes          20 bytes    20+ bytes    variable
```

When the sniffer reads a raw frame from the kernel, it gets ALL of that as one byte buffer. Its job is to walk the headers, learn where each boundary is, and pull the interesting fields out.

The OSI 7-layer model (simplified)

Layer	Name	Examples	What this sniffer does with it
1	Physical	Copper, fiber, radio	Out of scope (driver/firmware)
2	Data Link	Ethernet, 802.11 (WiFi)	Parses MAC addresses, EtherType, Radiotap
3	Network	IPv4, IPv6, ARP	Extracts source/dest IP, protocol field
4	Transport	TCP, UDP, ICMP	Extracts ports, TCP flags, ICMP type/code
5-7	Session/Pres./App	TLS, HTTP, DNS, EAPOL	Detects EAPOL signature; rest is payload

2.2 Raw sockets vs. application sockets

Normal programs use sockets like socket(AF_INET, SOCK_STREAM) which give them just the payload: the kernel handles all the headers. A raw socket, socket(AF_PACKET, SOCK_RAW, ETH_P_ALL), bypasses the kernel network stack and hands the program the full frame including the Ethernet header. This requires elevated privileges (CAP_NET_RAW).

2.3 Promiscuous mode and monitor mode

- Promiscuous mode: the NIC delivers every frame on the wire to the OS, not just those addressed to it. Necessary for sniffing a wired LAN or any traffic flowing through your local segment.
- Monitor mode (RFMON, WiFi only): the radio reports raw 802.11 frames including management/control frames the kernel would otherwise hide. It also disassociates from any AP, so the device loses internet connectivity while in this mode.

3. Protocols this sniffer parses

3.1 Ethernet II

Ethernet is the dominant Layer 2 protocol on wired LANs. The Ethernet II frame begins with destination and source MAC addresses (48 bits each) and a 16-bit EtherType identifying the upper-layer protocol.

Ethernet II header (14 bytes)

Dest MAC (6)	Src MAC (6)	EtherType (2)

- EtherType 0x0800 = IPv4
- EtherType 0x86DD = IPv6
- EtherType 0x0806 = ARP
- Values < 1536 (0x0600) are actually 802.3 length fields and indicate an older LLC frame; this sniffer filters those out as noise.

3.2 IPv4

IPv4 header (variable, min 20 bytes)

Ver+IHL	ToS (1)	TotLen (2)	ID (2)	Flags+Off (2)	TTL (1)	Proto (1)	Checksum (2)	Src IP (4)	Dst IP (4)

The IHL (Internet Header Length) is the number of 32-bit words in the header, so the header length in bytes is iph->ihl * 4. This is why the code multiplies by 4 in networkLayer.c. The Protocol byte tells you what comes next:

- 1 = ICMP (used by ping)
- 6 = TCP
- 17 = UDP
- 2 = IGMP (multicast group membership)

3.3 IPv6

IPv6 keeps the header simpler and a fixed length of 40 bytes. The 'Next Header' field plays the role of IPv4's Protocol field.

IPv6 header (fixed 40 bytes)

Ver/TC/Flow	Payload	Next Hdr	Hop Lim	Source address (16)	Destination address (16)

- Next Header 6 = TCP
- Next Header 17 = UDP
- Next Header 58 = ICMPv6

3.4 TCP

TCP gives connection-oriented, reliable, ordered byte streams. The header carries source/destination ports, sequence and acknowledgement numbers, and a flags byte. This sniffer surfaces the flags because they tell the story of a connection.

TCP header (min 20 bytes)

SrcPort (2)	DstPort (2)	SeqNum (4)	AckNum (4)	Off+Flags (2)	Window (2)	Checksum (2)	Urgent (2)

TCP flags packed into one byte

Bit	Flag	Meaning
0x02	SYN	Start a new connection
0x10	ACK	Acknowledging received data
0x01	FIN	Graceful close requested
0x04	RST	Abrupt reset
0x08	PSH	Deliver to app immediately, don't buffer
0x20	URG	Urgent pointer is meaningful

3.5 UDP

UDP is connectionless: just a tiny 8-byte header carrying source port, destination port, length, and checksum. Used by DNS, DHCP, QUIC, video calls, and -- in this very project -- to ship metadata from the C sniffer to the Python dashboard.

UDP header (8 bytes)

SrcPort (2)	DstPort (2)	Length (2)	Checksum (2)

3.6 ICMP and ICMPv6

ICMP carries control messages: echo request/reply (ping), destination unreachable, time exceeded (used by traceroute). The sniffer captures type and code, which together identify the message.

Type	Meaning (ICMP/IPv4)
0	Echo Reply (ping response)
3	Destination Unreachable
8	Echo Request (ping)
11	Time Exceeded (traceroute hop)

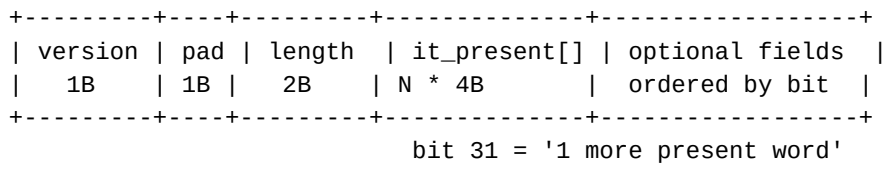
3.7 ARP

ARP (Address Resolution Protocol) translates IPv4 addresses to MAC addresses on a LAN. When you ping a host on your subnet, your machine first broadcasts 'who has 192.168.1.10?' and the owner replies 'me, at aa:bb:cc:dd:ee:ff'. This sniffer recognizes EtherType 0x0806 and labels these frames as ARP in the dashboard.

3.8 802.11 (WiFi) and the Radiotap wrapper

When a wireless card is in monitor mode, frames arrive prefixed with a Radiotap header that exposes physical-layer telemetry (signal strength, channel, modulation) that the radio observed when receiving the frame. The Radiotap header is variable-length: it advertises which optional fields it includes via a 32-bit 'present' bitmask. Different chipsets emit different field sets, so a robust parser must walk the bitmask.

Radiotap header structure


+---------+----+---------+--------------+------------------+
| version | pad | length | it_present[] | optional fields |
| 1B | 1B | 2B | N * 4B | ordered by bit |
+---------+----+---------+--------------+------------------+
bit 31 = '1 more present word'

Once past Radiotap, the 802.11 MAC header gives the frame type and subtype encoded in the Frame Control field:

Type	Meaning	Notable subtypes
0	Management	Beacon (8), Probe Request (4), Probe Response (5), Auth (11)
1	Control	RTS, CTS, ACK, Block ACK
2	Data	Data frames (often encrypted); EAPOL handshakes ride inside

3.9 EAPOL (WPA/WPA2 handshake)

When a device joins a WPA/WPA2 network, it performs a 4-way handshake with the access point. Each step is an EAPOL frame carried inside an 802.11 data frame. The sniffer detects EAPOL by scanning for the signature byte pattern 0xAA 0xAA 0x03 (LLC SNAP header) followed by EtherType 0x88 0x8E (EAPOL) and dumps matching frames to captured_handshake.cap in PCAP format for offline analysis.

NOTE: Capturing a handshake of someone else's network without permission may violate computer-misuse laws. This feature exists for auditing your own infrastructure.

4. Architecture

4.1 End-to-end pipeline

```
+--------+    +----------------+    +-----------------+
| Kernel | -> | C sniffer      | -> | Python dashboard|
|        |    | (raw socket +  |    | (UDP listener   |
|        |    |  zero-copy ring|    |  + rich TUI)    |
+--------+    +----------------+    +-----------------+
   ^                |                       |
   |                v                       v
   |       JSON metadata (UDP 127.0.0.1:5005)
   |
 NIC frames
```

4.2 Zero-copy capture (PACKET_MMAP)

A naive sniffer does recv() per packet, which copies bytes from kernel buffers into user-space memory. Under load this is expensive. Linux PACKET_MMAP allocates a ring buffer of fixed-size frames in kernel memory and maps it into the process's address space via mmap(). The kernel writes incoming packets directly to slots in this shared ring; the sniffer just reads them. No copy.

Ring buffer layout in this project

- Frame size: 2048 bytes (enough for any Ethernet frame plus the per-frame header).
- Block size: page-aligned (typically 4096 bytes on x86, holds two frames).
- Block count: 64. Total ring: 64 * 4096 = 256 KB.
- Frame count: 256 KB / 2048 = 128 slots.
- Each slot starts with a tpacket2_hdr whose tp_status byte is the kernel/user handshake.

Producer / consumer handshake

```
# Kernel sets tp_status |= TP_STATUS_USER  -> 'your turn'
# User reads frame, processes it
# User sets tp_status = TP_STATUS_KERNEL    -> 'all yours again'

while keep_running:
    if (hdr->tp_status & TP_STATUS_USER) == 0:
        poll(socket, 100ms)            # idle wait
        continue
    process(packet, hdr->tp_snaplen)   # decode
    hdr->tp_status = TP_STATUS_KERNEL  # release slot
    frame_idx = (frame_idx + 1) % N    # next slot
```

4.3 Threading model

The process runs two threads: the main thread spins the capture loop (producer) and a background logger thread drains a queue (consumer).

```
+---------------------+      mutex+condvar      +-----------------+
| main thread         |   ----------------->    | logger thread   |
```

```
| - polls ring        |   bounded queue (10k)    | - dequeues      |
| - parses packet     |                          | - prints text   |
| - enqueues message  |                          | - sends UDP JSON|
+---------------------+                          +-----------------+
```

Why a queue? The packet capture must stay responsive: printing or calling sendto() on every frame would block the producer and cause kernel drops. The queue absorbs short bursts. After the recent fix, the queue is bounded at 10,000 entries so a sustained overload drops messages instead of leaking memory.

4.4 IPC: why UDP for the dashboard?

- Decoupling: the C process and the Python process have independent lifecycles.
- Simplicity: send-and-forget; the C side does not stall if the dashboard is slow.
- Cheap: UDP has no connection setup; localhost loopback is essentially memcpy.
- Loss-tolerant: if a packet is dropped on the way to the dashboard, the next one arrives soon enough.

5. Module-by-module walkthrough

5.1 Source layout

```
Sniffer/
  main.c                  # Entry point, signal handling, top-level wiring
  CMakeLists.txt          # Build (C11, -Wall -Wextra -Wpedantic, pthread)
  build_and_run.sh        # Convenience: build + airmon-ng + run

  socket/
    rawSocket.{c,h}       # AF_PACKET socket, bind, promiscuous toggle

  core/
    mmapSniffer.{c,h}     # PACKET_MMAP ring, poll loop, frame dispatch
    packetParser.{c,h}    # Dispatcher: managed vs monitor mode
    managedMode.{c,h}     # Ethernet -> IP -> TCP/UDP/ICMP path
    monitorMode.{c,h}     # Radiotap -> 802.11 path + EAPOL detection

  layers/
    ethernetLayer.{c,h}   # Pulls MACs and EtherType
    networkLayer.{c,h}    # Pulls IPv4/IPv6 src/dst + protocol/next-header
    transportLayer.{c,h}  # Pulls ports + TCP flags + ICMP type/code

  common/
    Types.h               # PacketMetadata struct shared between C modules
    logger.{c,h}          # Thread-safe queue + worker thread
    udp_sender.{c,h}      # JSON encode + sendto() to dashboard port

  python/
    app.py                # Rich Live loop
    data_listener.py      # Non-blocking UDP recv, deque + Counter stats
    ui_renderer.py        # Layout, table, color-coding
```

5.2 main.c

main.c is intentionally tiny. It validates argv, installs a SIGINT handler that flips the global keep_running flag, asks the kernel whether the interface is in monitor mode (via SIOCGIFHWADDR returning ARPHRD_IEEE80211_RADIOTAP), opens the raw socket, configures the zero-copy ring, runs the capture loop, then tears everything down in reverse order on exit.

5.3 socket/rawSocket.c

- socket(AF_PACKET, SOCK_RAW, htons(ETH_P_ALL)) -- raw frames, all protocols.
- SIOCGIFINDEX to resolve the interface name to an integer index.
- bind() a sockaddr_ll so we only see frames from that one interface.
- SIOCGIFFLAGS / SIOCSIFFLAGS to enable IFF_PROMISC, and clear it on close.

5.4 core/mmapSniffer.c

Sets PACKET_VERSION to TPACKET_V2, computes a frame/block geometry, calls

setsockopt(PACKET_RX_RING) to ask the kernel to allocate it, then mmap()s the ring into user space. The capture loop polls the current frame's tp_status byte. After the recent fix, the TP_STATUS_LOSING warning is throttled to at most one per second with an aggregate count, so a stuck/lossy state cannot flood the logger.

5.5 core/packetParser.c

Thin dispatcher: chooses managed or monitor based on a flag set from main.c. After parsing, applies a feedback-loop guard that drops metadata describing the sniffer's own UDP-5005 traffic. Without this guard, sniffing the loopback (or any interface that carries the dashboard packets) creates exponential amplification.

5.6 core/monitorMode.c

- Validates the Radiotap header length advertised at offset 2 (LE u16, read with memcpy to avoid unaligned-pointer UB).
- Walks the it_present bitmask, applying each field's natural alignment relative to the radiotap header start, and extracts channel (bit 3) and signal dBm (bit 5).
- Reads the 802.11 frame control, decodes type and subtype, pulls source and destination MAC.
- For management frames, walks tagged parameters looking for the SSID tag (id 0). SSID is capped to sizeof(meta->ssid)-1 and explicitly NUL-terminated.
- For data frames, scans for the EAPOL byte signature and, on match, writes the frame to captured_handshake.cap using the PCAP file format with link-type DLT_IEEE802_11_RADIO (127).

5.7 layers/*

- ethernetLayer: 14 bytes -> src/dst MAC + ntohs(ether_type).
- networkLayer: looks at the top nibble (version) to dispatch IPv4 vs IPv6, uses inet_ntop to render addresses to text, multiplies IHL by 4 for the IPv4 header length.
- transportLayer: pulls ports with ntohs, repacks TCP bitfield flags into a single byte using the legacy 0x01/0x02/0x04/0x08/0x10/0x20 layout so the dashboard can decode them with simple bit tests.

5.8 common/logger.c

Implements a producer-consumer queue protected by a pthread mutex and signalled via a condvar. The worker thread blocks on the condvar when the queue is empty, drains it otherwise. Two enqueue paths exist: log_message for printf-style text and log_packet for PacketMetadata structs. Both honor the LOGGER_QUEUE_MAX cap (10,000 entries) and drop-with-counter on overflow.

5.9 common/udp_sender.c

Initializes a UDP socket targeting 127.0.0.1:DASHBOARD_UDP_PORT (5005) and serializes each PacketMetadata into a compact JSON document with snprintf. Sends with sendto(); the return value is intentionally ignored because metadata loss is acceptable.

5.10 Python dashboard (python/*)

- app.py runs a rich.live.Live loop refreshing four times per second.
- data_listener.py opens a non-blocking UDP socket on 5005, fetches all available datagrams each

tick, decodes JSON, and updates a Counter for top talkers, a Counter for protocols, a total byte count, and a deque of the last 25 packets for the live table.

- ui_renderer.py defines the Layout (header / packets+stats / footer), creates the live packet table with color coding per protocol, and renders the right-hand stats panel.

6. The Dashboard UI

6.1 Screen layout

```
+-----------------------------------------------------------+
|                  WiFi & Network Dashboard                 |   <- header
+-------------------------------------+--------------------+
| Time  Proto  Source  Dest  Info  Sig | Top Sources        |
| ...                                  | 192.168.1.5 : 42   |
| ...   (rolling live packet table)    |                    |
| ...                                  | Breakdown          |
| ...                                  | TCP : 120          |
|                                      | UDP : 60           |
|                                      | DNS : 5            |
|                                      |                    |
|                                      | Total: 142.3 KB    |
+-------------------------------------+--------------------+
|       Running on Localhost:5005  -  Ctrl+C to exit        |  <- footer
+-----------------------------------------------------------+
```

6.2 Packet table columns

Column	Meaning
Time	Local time-of-day when the packet was received by the dashboard
Proto	Color-coded protocol label (TCP, UDP, ICMP, ARP, BEACON, PROBE, DATA, EAPOL...)
Source	ip:port (with service name when known) or MAC for 802.11/ARP
Destination	ip:port or MAC
Info	Protocol-specific detail: TCP flags, DNS/DHCP/QUIC heuristic, SSID + channel
Signal/Type	WiFi: dBm signal strength (green/yellow/red). Wired: link label

6.3 Color coding cheat-sheet

Protocol	Color	Why
TCP	Bold blue	Bulk of the traffic on most networks
UDP	Bold orange	Less common; flags DNS/DHCP/QUIC heuristically
ICMP / ICMPv6	Cyan	Diagnostic traffic, ping, traceroute
ARP	Bold magenta	Layer-2 address resolution
BEACON (802.11)	Bold green	Access points advertising SSID
PROBE (802.11)	Bold yellow	Clients actively searching for known networks
DATA (802.11)	Dim white	Encrypted wireless payload
EAPOL	Bold red, blinking	Key exchange captured -- audit/security signal

6.4 Stats panel

- Top Sources -- ranks the chattiest IPs (or MACs in 802.11 mode) by packet count.
- Breakdown -- per-protocol packet counts since the dashboard started.
- Total -- cumulative bytes seen, rendered in kilobytes for readability.

7. Operation modes

7.1 Managed mode

Standard mode. The interface stays associated with its access point or wired LAN, internet keeps working, and the sniffer sees frames the OS would normally see (plus, with promiscuous mode on, anything else on the local segment that reaches the NIC). This is the right choice for:

- Debugging local TCP/UDP issues.
- Watching DNS queries from this device.
- General observability without disturbing connectivity.

7.2 Monitor mode (RFMON)

WiFi-only. The radio is reconfigured to deliver raw 802.11 frames including management/control traffic that the host would otherwise filter. Requires a WiFi chipset/driver that supports it (most modern chipsets do). build_and_run.sh uses airmon-ng start <iface> to flip the radio and then channel-hops 1..13 to scan all 2.4 GHz channels.

NOTE: Monitor mode disconnects your WiFi. To capture a specific exchange (e.g. a WPA handshake), lock the radio to that AP's channel with 'iw dev <iface> set channel <N>' instead of hopping.

8. Build and run

8.1 Dependencies

```
# System packages (Debian/Ubuntu)
sudo apt update
sudo apt install build-essential cmake aircrack-ng python3-pip libcap2-bin

# Python (rich for the dashboard)
pip install --user rich
# or in a venv:
python3 -m venv .venv && . .venv/bin/activate && pip install rich
```

8.2 Build

```
mkdir -p build && cd build
cmake ..
make
# produces ./build/Sniffer
```

8.3 Grant capabilities (no sudo needed at run time)

Instead of running the binary as root, grant it just the two Linux capabilities it needs: open raw sockets and toggle interface flags.

```
sudo setcap cap_net_raw,cap_net_admin+ep ./build/Sniffer
```

NOTE: Linux capabilities are wiped when a file is replaced. Re-run setcap after every rebuild, or have your build script do it automatically.

8.4 Run

```
# Terminal 1: capture
./build/Sniffer wlp2s0          # or any other interface name

# Terminal 2: dashboard
cd python && python3 app.py     # full-screen TUI, Ctrl+C to exit
```

8.5 Or use the automation script

```
chmod +x build_and_run.sh
./build_and_run.sh wlp2s0
# Prompts for managed vs monitor mode, builds, starts sniffer + dashboard,
# cleans up promiscuous/monitor mode on exit.
```

9. Recent fixes (branch: fix/runtime-stability)

A round of bug-hunting and runtime testing surfaced six issues. All are fixed on the fix/runtime-stability branch. Summary:

#	Bug	Fix
1	TP_STATUS_LOSING warning logged per-frame -> flood (~128k/sec)	Throttle to one aggregated warning per second
2	Logger queue unbounded -> OOM risk under load	Cap at 10,000 entries; drop+count overflow
3	Dashboard UDP re-captured -> exponential amplification on lo	Skip metadata with src/dst port 5005 in non-monitor mode
4	Unaligned *(uint16_t*)ptr reads (undefined behaviour in C11)	Replace with memcpy in Radiotap and 802.11 parsing
5	Hardcoded Radiotap offsets only worked for one chipset class	Walk it_present bitmask with per-field alignment per spec
6	SSID copy used magic 32; potential for off-by-one ambiguity	Use sizeof(meta->ssid)-1 and explicit NUL terminator

9.1 Measured impact (5 second test on loopback)

Metric	Before	After
Stdout log volume	31 MB	176 bytes
Dashboard UDP volume	280 MB	105 KB
Ring-buffer warnings	639,978	0
Compiler warnings	1 (unused fn)	0

10. Glossary

Term	Definition
AF_PACKET	Linux socket family that gives you raw L2 frames directly from the NIC
Capability	Fine-grained Linux privilege; CAP_NET_RAW allows raw sockets without full root
EAPOL	Extensible Authentication Protocol over LAN; carries WPA/WPA2 4-way handshake
EtherType	16-bit Ethernet field naming the next protocol (0x0800=IP, 0x86DD=IPv6, 0x0806=ARP)
IHL	Internet Header Length in 32-bit words; iph->ihl*4 = bytes
MMAP	Memory-mapped I/O; shares a region between kernel and user space without copies
Promiscuous	NIC mode that delivers every frame seen on the wire, not just those addressed to it
Radiotap	Variable-length header prepended to 802.11 frames in monitor mode with PHY info
RFMON	Radio Frequency Monitor; another name for WiFi monitor mode
RSSI	Received Signal Strength Indication; reported in dBm (closer to 0 = stronger)
SSID	Service Set Identifier; the human-readable name of a WiFi network
TPACKET_V2	Version of the AF_PACKET ring-buffer ABI used by this project
TUI	Text User Interface; full-screen, keyboard-driven UI inside a terminal

11. References

- Linux kernel: Documentation/networking/packet_mmap.rst (PACKET_MMAP ABI)
- Radiotap specification: https://www.radiotap.org
- IEEE 802.11: https://standards.ieee.org
- RFC 791 - IPv4
- RFC 8200 - IPv6
- RFC 793 - TCP
- RFC 768 - UDP
- RFC 792 - ICMP
- RFC 4443 - ICMPv6
- PCAP file format: https://wiki.wireshark.org/Development/LibpcapFileFormat
- rich (Python): https://rich.readthedocs.io